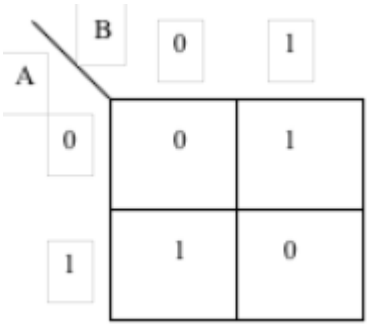
Università degli Studi di Trento Corso di Programmazione I – E3: 145430/145935 4° Appello – 09/07/2019 – a.a. 2018/2019	**PC N°**
COGNOME ..... **NOME.....** **MATRICOLA**	
RITIRATO	**FIRMA PER RITIRO.....**

Completare le voci **Cognome**, **Nome** e **Matricola**, se tali voci saranno mancanti o non-leggibili elaborato **NON** verrà corretto.
NON è consentito utilizzo di alcun strumento cartaceo, elettronico, informatico etc. durante la prova.
Fa parte della valutazione la leggibilità del codice C/C++.
Scrivere le risposte su questo foglio e riconsegnare. I risultati verranno pubblicati su DOL
Tempo a disposizione: 120 minuti.

PARTE TEORICA: 13 PUNTI – PUNTEGGIO MINIMO 6 PUNTI

[Domanda 1 – punti 3] Si definisca in C++ una funzione di nome *TVL* che prende in input 2 valori logici A, B e restituisce un valore logico come definito nella tabella seguente (dove 0 indica FALSE e 1 indica TRUE)



Si realizzi tale funzione mediante un'opportuna combinazione delle funzioni base OR e NOT (e non altre!).

[Domanda 2 - punti 3] Data la funzione ricorsiva:

```
int calcola(int n1, int n2) {
     int a, b;
     a = n1 + 2;
     b = n2 - 3;
     printf("a=%d b=%d\n",a,b);
     if ( 0 ){
          return a;
     }else{
          return calcola(a,b);
     }
 }
```

Descrivere cosa viene stampato nelleprime 4 invocazioni della funzione in seguito all'invocazione: `calcola(8,9);`
Come si deve modificare la condizione affinché la ricorsione termini sempre?

[Domanda 3 – punti 1] Una grandezza che può assumere 354 valori distinti può essere codificata con:

SI ☐ NO ☐ 1 byte
SI ☐ NO ☐ 2 byte
SI ☐ NO ☐ 4 byte
SI ☐ NO ☐ 10 byte
SI ☐ NO ☐ 1 kilobyte
SI ☐ NO ☐ mezzo byte

[Domanda 4 - punti 4]
[A] Data la seguente struttura:

```
typedef struct Tnodo{
  int dato;
  Tnodo *next, *prev;
} Tnodo;
```

scrivere una funzione che cancella dalla lista doppiamente concatenata tutti gli elementi con campo dato maggiore di una soglia s. La funzione riceve il puntatore alla testa della lista l e la soglia s e restituisce il (nuovo) puntatore alla lista.
Considerare i casi in cui gli elementi da cancellare si trovano all'inizio e/o alla fine della lista.
Mostrare una invocazione della funzione dichiarando e passando gli opportuni parametri.

[Domanda 4 - punti 2] Come mai i calcoli a virgola mobile sono così imprecisi? Dare una spiegazione dettagliata della problematica.
.

PARTE PRATICA: 20 PUNTI – PUNTEGGIO MINIMO 10 PUNTI

[A - punti 5] *Inserire i seguenti tipi in un file* ***dati.h*** **e implementare i metodi di stampa ed i costruttori:**

```
#define DIM 2
typedef enum Tpianta { MONTANO, COLLINARE, TROPICALE } Tpianta;
typedef struct Tforesta{
  char nomeForesta[20]; unsigned int numeroPiante;
  float altezzaMedia;
  Tpianta tipoPianta;
  Tforesta ()   { /*da implementare*/ }
  void stampa() { /*da implementare*/ }
} Tforesta;
typedef struct TcodaFIFO {
  int n, dim, head, tail;
  Tforesta *s;
  TcodaFIFO ()     { /*da implementare*/ }
  TcodaFIFO (int x){ /*da implementare*/ }
  void stampa()       { /*da implementare*/ }
} TcodaFIFO;
// Inserimento e cancellazione dalla testa (head)
```

LEGENDA struct TcodaFIFO
n = numero elementi nella coda
dim = dimensione max coda
head = posizione elemento in testa
tail = posizione elemento in coda
s = vettore elementi

[B - punti 1] *Modificare il file* ***main.cpp*** *includendo il file* ***dati.h*** *ed inserendo il seguente codice nel* ***main****:*

```
TcodaFIFO* ff = new TcodaFIFO(10);
Tforesta f;
for(int i=0; i<5; i++) { initForesta(&f); addForesta(ff,f); }
stampaForeste(ff);
printf("piante %d\n", rimuoviESalvaForeste(ff, COLLINARE));
```

[C - punti 3] Implementare una funzione **initForesta** che riceve in input l'indirizzo di una variabile di tipo **Tforesta f** e la inizializza con:

- campo **nomeForesta** letta da tastiera;
- campo **numeroPiante** valore letto da tastiera, tra 1000 e 9999, con controllo input.
- campo **altezzaMedia** valore casuale tra 100,00 e 2000,00;
- campo **tipoPianta** valore casuale tra MONTANO, COLLINARE, TROPICALE.

[D - punti 4] Implementare una funzione **addForesta** che riceve come input una variabile di tipo **TcodaFIFO *foreste**, ed una variabile **f** di tipo **Tforesta**. La funzione inserisce nella coda FIFO l'elemento **f**. (Inserimento in testa – head)

[E - punti 2] Implementare una funzione **stampaForeste** che riceve come input una variabile di tipo **TcodaFIFO *foreste** stampa il contenuto delle liste utilizzando opportunamente il metodo **stampa** di **TcodaFIFO** nella forma (per l'enum stampare etichetta in modo opportuno):

```
nomeForesta (tipoPianta) altezzaMedia numeroPiante
```

[F - punti 5] Implementare una funzione **rimuoviESalvaForeste** che riceve come input una variabile di tipo **TcodaFIFO *foreste** e **Tpianta tp.** La funzione elimina tutti gli elementi dalla coda FIFO (cancellazione da testa – head) e restituisce il numero di piante con tipoPianta pari a **tp** e salva tali elementi in un file denominato **"piante.txt"** nella forma:

```
nomeForesta numeroPiante
```

È possibile creare strutture, funzioni, metodi aggiuntivi per realizzare gli esercizi proposti.
Il codice commentato NON verrà corretto!